

AI SCIENTIST

FASE 2 | AULA 01 -

INTRODUÇÃO AO NOSQL: CONCEITOS E DIFERENÇAS EM RELAÇÃO AO MODELO RELACIONAL

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

SUMÁRIO

O QUE VEM POR AÍ? 3
HANDS ON 4
SAIBA MAIS 5
MERCADO, CASES E TENDÊNCIAS 20
O QUE VOCÊ VIU NESTA AULA? 22
REFERÊNCIAS 23

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

O QUE VEM POR AÍ?

A compreensão dos fundamentos do **NoSQL** representa uma mudança relevante na forma como os sistemas de dados são concebidos em ambientes modernos. Mais do que uma alternativa ao modelo relacional, o NoSQL reflete uma resposta arquitetural às demandas de **escala, distribuição e flexibilidade**, características centrais de aplicações digitais contemporâneas.

Enquanto o modelo relacional foi projetado para contextos com dados estruturados, schemas estáveis e consistência imediata, soluções NoSQL emergem para lidar com **volumes massivos de dados**, padrões de acesso imprevisíveis e ambientes distribuídos. Nesse cenário, decisões de modelagem deixam de priorizar normalização e passam a ser orientadas pelos **requisitos de leitura e escrita das aplicações**, impactando diretamente desempenho e disponibilidade.

Outro ponto fundamental é a forma como o NoSQL trata a consistência dos dados. Ao operar sob o princípio da **consistência eventual**, esses sistemas assumem que falhas e latências fazem parte do ambiente distribuído, privilegiando alta disponibilidade e tolerância a partições. Essa escolha não representa fragilidade, mas sim um trade-off consciente, alinhado às necessidades de sistemas em larga escala.

Além disso, a flexibilidade de schema amplia a capacidade de evolução dos sistemas, permitindo adaptações rápidas sem os custos operacionais associados a migrações complexas. Em arquiteturas contemporâneas, é comum a coexistência de bancos relacionais com soluções NoSQL como MongoDB, Redis e Neo4j, evidenciando uma abordagem pragmática e orientada ao problema.

Nesse contexto, o domínio conceitual do NoSQL não se limita à tecnologia, mas envolve uma visão arquitetural capaz de equilibrar consistência, flexibilidade, desempenho e custo em ambientes distribuídos e dinâmicos.

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

HANDS ON

INTRODUÇÃO AO NOSQL COM DATABRICKS

O objetivo deste Hands On é proporcionar uma experiência prática controlada sobre os conceitos fundamentais do NoSQL, utilizando o Databricks Free (Community Edition) como ambiente de experimentação.

Embora o Databricks não seja um banco NoSQL transacional, sua capacidade de trabalhar nativamente com dados semiestruturados (JSON), schemas flexíveis e estruturas aninhadas o torna um excelente laboratório para compreender o paradigma NoSQL, especialmente o modelo orientado a documentos.

Durante a atividade, o foco não estará na ferramenta, mas sim em como os dados são estruturados, como as consultas são realizadas e quais decisões arquiteturais mudam em relação ao modelo relacional.

Importante: todo o Hands On será realizado em Python (PySpark) dentro de um notebook Databricks.

Pré-requisitos

Antes de iniciar, certifique-se de que você possui:

- Conta gratuita no Databricks Community Edition;
- Acesso a um navegador (Chrome, Firefox ou Edge);
- Conhecimento básico de Python (variáveis e listas);
- Noções introdutórias de bancos de dados relacionais.

Veja o passo a passo do Hands On na seção “Saiba mais”.

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

SAIBA MAIS

Passo a passo — Preparando o Ambiente

Passo 1 — Acessar o Databricks Free

Acesse: https://community.cloud.databricks.com, faça login ou crie uma conta gratuita. Após o login, você será direcionado ao Workspace.

Passo 2 — Criar um Notebook

No menu lateral esquerdo, clique em Workspace. Em seguida, clique com o botão direito → Create → Notebook. Depois, nomeie o notebook, por exemplo: **HandsOn_NoSQL_Databricks**

Selecione:

- Language: Python;
- Cluster: Starter (padrão do Community Edition).

Passo 3 — Validar o Ambiente

Execute a célula abaixo para validar o ambiente Spark:
python

```
spark
```

Se o objeto `SparkSession` for exibido, o ambiente está pronto.

Parte 1 — Referência Relacional (Comparação Conceitual)

Objetivo

Demonstrar como um mesmo problema é tradicionalmente resolvido no modelo relacional, utilizando múltiplas tabelas e operações de JOIN.

Criando DataFrames Relacionais

python

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

```
usuarios = spark.createDataFrame([
    (1, "Ana Souza"),
    (2, "Carlos Lima")
], ["usuario_id", "nome"])

cursos = spark.createDataFrame([
    (101, "Data Engineering"),
    (102, "Big Data")
], ["curso_id", "nome"])

progresso = spark.createDataFrame([
    (1, 101, 65),
    (2, 102, 40)
], ["usuario_id", "curso_id", "progresso"])
```

Código fonte 1 – Exemplo de código fonte python

Fonte: Elaborado pelo autor (2026)

Consulta com JOIN python:

```
resultado_relacional = usuarios \
    .join(progresso, "usuario_id") \
    .join(cursos, "curso_id")

display(resultado_relacional)
```

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

Código fonte 2 – Exemplo de código fonte python
Fonte: Elaborado pelo autor (2026)

Ponto de reflexão: quantos JOINs foram necessários? Como essa abordagem se comporta em grande escala?

Parte 2 — Modelagem NoSQL com Documentos (JSON)

Objetivo

Representar o mesmo cenário utilizando documentos, aplicando os princípios do NoSQL.

Criando dados no formato documento python

```
dados_nosql = [
    {
        "usuario_id": 1,
        "nome": "Ana Souza",
        "cursos": [
            {
                "curso_id": 101,
                "nome": "Data Engineering",
                "progresso": 65,
                "modulos": [
                    {"modulo": "Introdução", "concluido": True},
                    {"modulo": "NoSQL", "concluido": False}
```

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

```
                    ]
                }
            ]
        },
        {
            "usuario_id": 2,
            "nome": "Carlos Lima",
            "cursos": [
                {
                    "curso_id": 102,
                    "nome": "Big Data",
                    "progresso": 40
                }
            ]
        }
    ]
```

Código fonte 3 – Exemplo de código fonte python

Fonte: Elaborado pelo autor (2026)

Criando o DataFrame NoSQL python:

```
df_nosql = spark.createDataFrame(dados_nosql)
df_nosql.printSchema()
display(df_nosql)
```

Código fonte 4 – Exemplo de código fonte python

Fonte: Elaborado pelo autor (2026)

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

Observação: Schema aninhado, estrutura flexível, dados acessados juntos armazenados juntos.

Parte 3 — Consulta Orientada a Documento

Consulta direta (sem JOIN) python:

```
df_nosql.select(
    "nome",
    "cursos.nome",
    "cursos.progresso"
).show(truncate=False)
```

Código fonte 4 – Exemplo de código fonte python
Fonte: Elaborado pelo autor (2026)

Quando necessário: `explode` python

```
from pyspark.sql.functions import explode

df_explodido = df_nosql \
    .withColumn("curso", explode("cursos")) \
    .select(
        "nome",
        "curso.nome",
        "curso.progresso"
    )

display(df_explodido)
```

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

Código fonte 5 – Exemplo de código fonte python
Fonte: Elaborado pelo autor (2026)

Discussão: o `explode` substitui JOINs? Quando ele deve ser evitado?

Parte 4 — Simulando Atualizações NoSQL

Objetivo

Demonstrar como alterações em documentos impactam o processamento.

```
from pyspark.sql.functions import expr


df_atualizado = df_nosql.withColumn(
    "cursos",
    expr("""
      transform(
        cursos,
        c -> IF(
          c.curso_id = 102,
          named_struct(
            'curso_id', c.curso_id,
            'nome', c.nome,
            'progresso', 60
          ),
          c
        )
      )
    """)
```

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

```
display(df_atualizado)
```

Código fonte 6 – Exemplo de código fonte python
Fonte: Elaborado pelo autor (2026)

Reflexão: por que atualizações são mais custosas em NoSQL? Quando isso deixa de ser um problema?

Parte 5 — Simulando Chave-Valor (Cache)

```
cache_progresso = df_explodido \
    .selectExpr(
        "concat('progresso:user:', nome) as chave",
        "progresso as valor"
    )

display(cache_progresso)
```

Código fonte 7 – Exemplo de código fonte python
Fonte: Elaborado pelo autor (2026)

Conceito aplicado: estrutura chave → valor. Acesso direto, ideal para cache.

O Databricks permite experimentar conceitos NoSQL de forma segura e controlada. Reforçando que NoSQL é, antes de tudo, uma mudança de mentalidade arquitetural.

A evolução dos bancos de dados e o contexto do NoSQL

Durante décadas, o **modelo relacional** foi a principal solução para armazenamento de dados estruturados. Sua proposta, baseada em tabelas, chaves primárias, chaves estrangeiras e normalização, trouxe ganhos significativos de **consistência, integridade e padronização**. Esses atributos foram fundamentais

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

para o crescimento dos sistemas corporativos, especialmente em ambientes onde transações confiáveis eram essenciais.

Entretanto, o cenário tecnológico mudou profundamente. A popularização da internet, o surgimento de aplicações globais, dispositivos móveis, redes sociais e sensores conectados geraram um volume de dados sem precedentes. Além do volume, aumentou também a **variedade dos dados**, que passaram a incluir textos, eventos, logs, cliques, interações e informações sem estrutura fixa.

Nesse novo contexto, tornou-se cada vez mais difícil sustentar todas as demandas exclusivamente com bancos relacionais tradicionais. O **NoSQL surge como resposta a esse novo paradigma**, não para substituir o modelo relacional, mas para **complementá-lo**, atendendo a cenários onde escala, flexibilidade e disponibilidade são mais relevantes do que a consistência imediata.

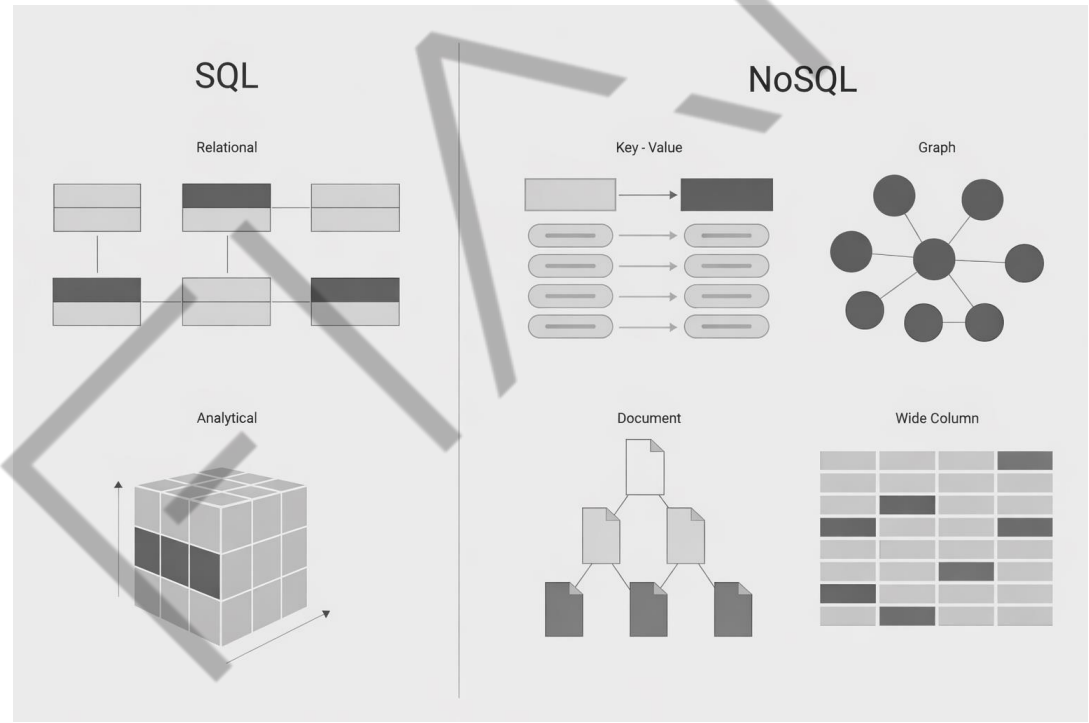


Figura 1 – O que é o NoSQL?

Fonte: Elaborado pelo autor (2026)

O termo **NoSQL** é frequentemente mal interpretado. Ele não significa “sem SQL” ou “contra SQL”. Na prática, NoSQL significa **“Not Only SQL”**, indicando que existem outras abordagens possíveis para persistência de dados além do modelo relacional.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

NoSQL não é:

- Um padrão único;
- Uma linguagem específica;
- Uma tecnologia isolada.

NoSQL é um **conjunto de abordagens arquiteturais** que compartilham características como:

- Escalabilidade horizontal;
- Tolerância a falhas;
- Flexibilidade de schema;
- Modelagem orientada ao acesso.

Essas características tornam o NoSQL especialmente adequado para aplicações distribuídas, ambientes em nuvem e sistemas que lidam com grandes volumes de dados heterogêneos.

Sistemas distribuídos e escalabilidade horizontal

Uma das principais diferenças entre o modelo relacional clássico e o NoSQL está na **forma de escalar**. Tradicionalmente, bancos relacionais escalam verticalmente, ou seja, aumentando recursos de um único servidor. Essa abordagem tem limites físicos e custos elevados.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

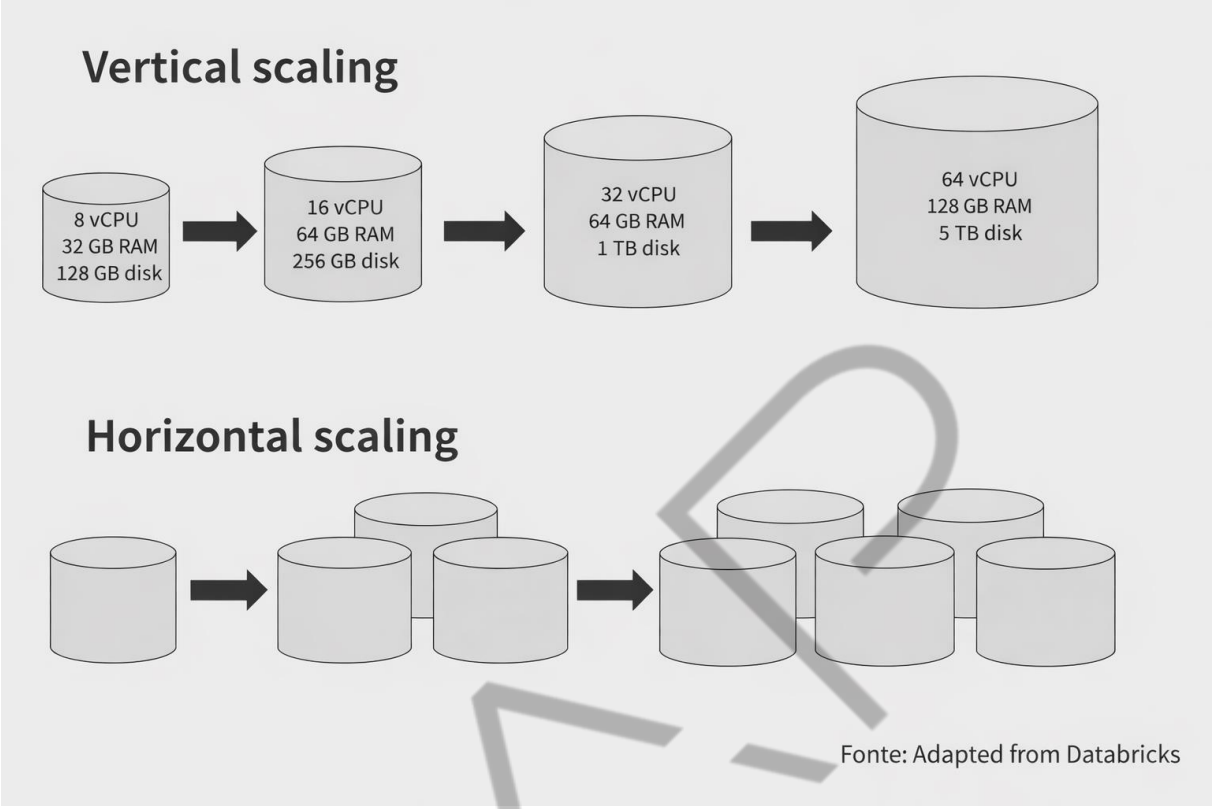


Figura 2 – NoSQL – Escala vertical e horizontal

Fonte: Databricks, adaptado pelo autor (2026)

O NoSQL, por outro lado, foi concebido para **escalabilidade horizontal**, permitindo a adição de novos nós ao sistema. Para viabilizar esse modelo, os dados são distribuídos por meio de **Sharding** (particionamento dos dados) e **Replicação** (cópias para alta disponibilidade)

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

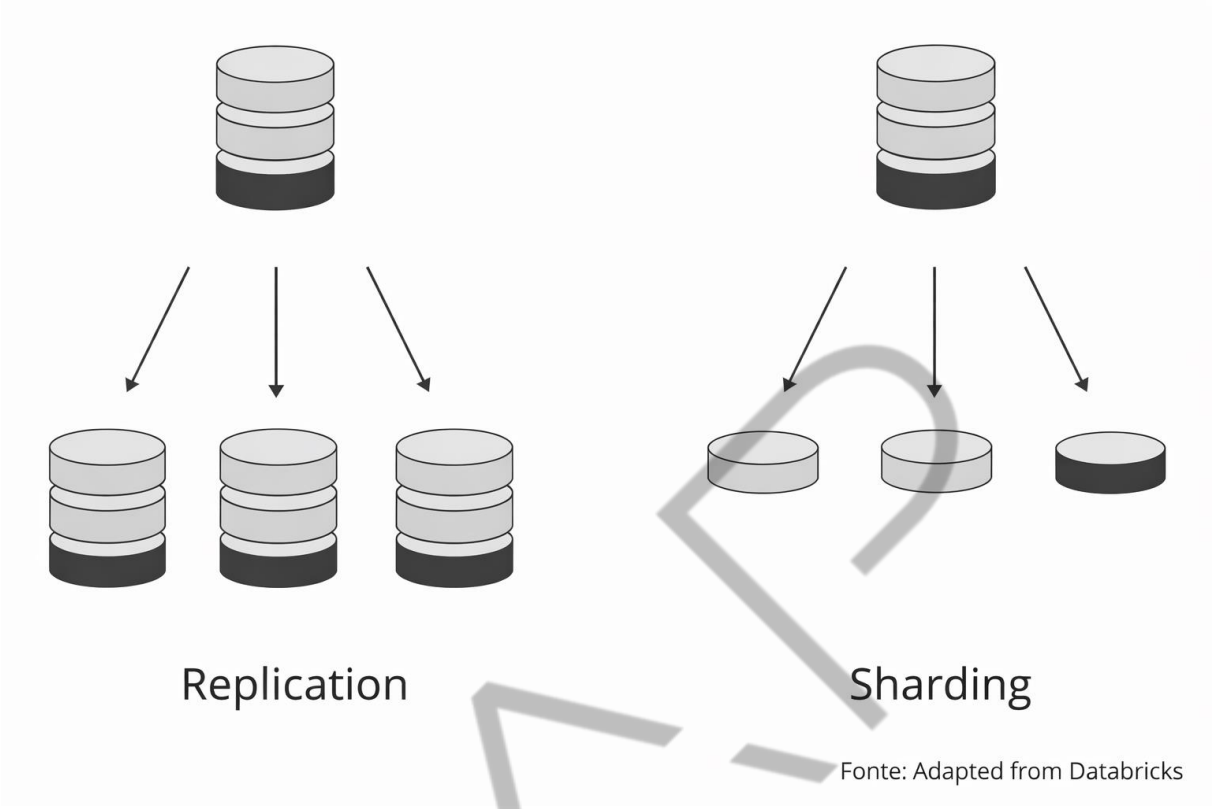


Figura 3 – NoSQL – Replication e Sharding

Fonte: Databricks, adaptado pelo autor (2026)

Essa arquitetura aumenta a resiliência e a capacidade de atendimento, mas introduz novos desafios relacionados à **consistência e sincronização** dos dados — aspectos que são tratados de forma diferente do modelo relacional.

O Teorema CAP e suas implicações práticas

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

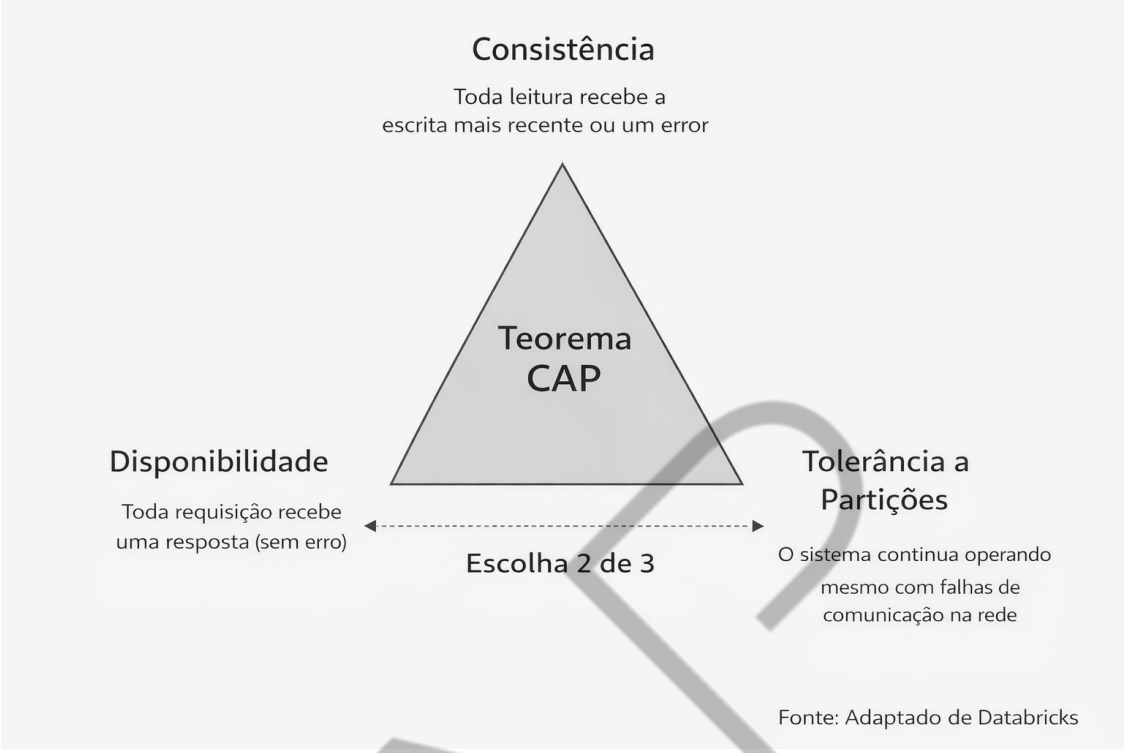


Figura 4 – NoSQL – Teorema CAP

Fonte: Databricks, adaptado pelo autor (2026)

O **Teorema CAP** é um dos pilares conceituais para compreender bancos NoSQL. Ele estabelece que, em sistemas distribuídos, não é possível garantir simultaneamente:

- **Consistência;**
- **Disponibilidade;**
- **Tolerância a partições.**

Como falhas de rede são inevitáveis, sistemas modernos assumem a tolerância a partições como obrigatória. A decisão passa então a ser entre **consistência forte** e **alta disponibilidade**.

Muitos bancos NoSQL priorizam disponibilidade, adotando a **consistência eventual**. Isso significa que, por um período limitado, diferentes nós podem apresentar valores distintos, mas o sistema converge para um estado consistente ao longo do tempo.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

Esse comportamento é aceitável — e até desejável — em muitos cenários, como redes sociais, sistemas de recomendação e aplicações de grande escala.

Modelos de dados NoSQL e suas aplicações

Diferentemente do modelo relacional, o NoSQL oferece múltiplos modelos de dados:

- **Documentos**: dados hierárquicos e auto-descritivos. Exemplo: MongoDB.
- **Chave-Valor**: acesso direto e alta performance. Exemplo: Redis.
- **Colunar (Wide-Column)**: escrita intensiva e alta escala. Exemplo: Apache Cassandra.
- **Grafos**: relações complexas e conectividade. Exemplo: Neo4j.



Comparação: Banco de Dados Relacional × NoSQL

Aspecto	Banco de Dados Relacional (SQL)	Banco de Dados NoSQL
Modelo de Dados	Tabelas (linhas e colunas)	Variável: documentos, chave-valor, grafos, colunas
Estrutura (Schema)	Rígido e previamente definido	Flexivel ou schema-less
Normalização	Alta (3FN, BCNF)	Baixa ou inexistente (desnormalização consciente)
Relacionamentos	Explicitos via chaves estrangeiras	Implicitos ou embutidos nos dados
Consultas	SQL padrão	APIs especificas, DSLs ou SQL-like
JOINs	Amplamente utilizados	Evitados ou inexistentes
Escalabilidade	Vertical (scale-up)	Horizontal (scale-out)
Distribuição	Limitada e complexa	Nativa para ambientes distribuidos
Consistência	Forte (ACID)	Eventual ou configurável (BASE)
Performance	Excelente para transações complexas	Limitadas ou especificas
Latência	Geralmente maior em ambientes	Excelente para leituras e escritas em escala
Flexibilidade	Baixa (migrações custosas)	Baixa latência
Casos de Uso	ERP, sistemas financeiros, constábeis	Big Data, APIs, microserviços, IoT
Exemplos	PostgreSQL, MySQL, Oracle	MongoDB, Redis, Cassandra, Neo4j

Observação Importante: Relacional e NoSQL não são concorrentes diretos, mas tecnologias complemen··
→ Arquiteturas modernas adotam Polyglot Persistence, escolhendo e combinando tecnologias conforme as neces

Figura 5 – Banco de dados relacional x NoSQL

Fonte: Databricks, adaptado pelo autor (2026)

Cada modelo resolve melhor um tipo específico de problema, reforçando que **não existe uma solução única**.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

Modelagem orientada ao acesso e desnormalização

No modelo relacional, a modelagem parte da normalização e da eliminação de redundâncias. No NoSQL, o ponto de partida é diferente: **as consultas da aplicação**.

A pergunta central passa a ser: “Como os dados serão lidos e escritos?”.

Essa abordagem leva à **desnormalização consciente**, onde dados são duplicados para reduzir leituras e evitar JOINs. Embora isso aumente o custo de escrita, melhora significativamente o desempenho de leitura, especialmente em sistemas distribuídos.

NoSQL em arquiteturas modernas e Polyglot Persistence

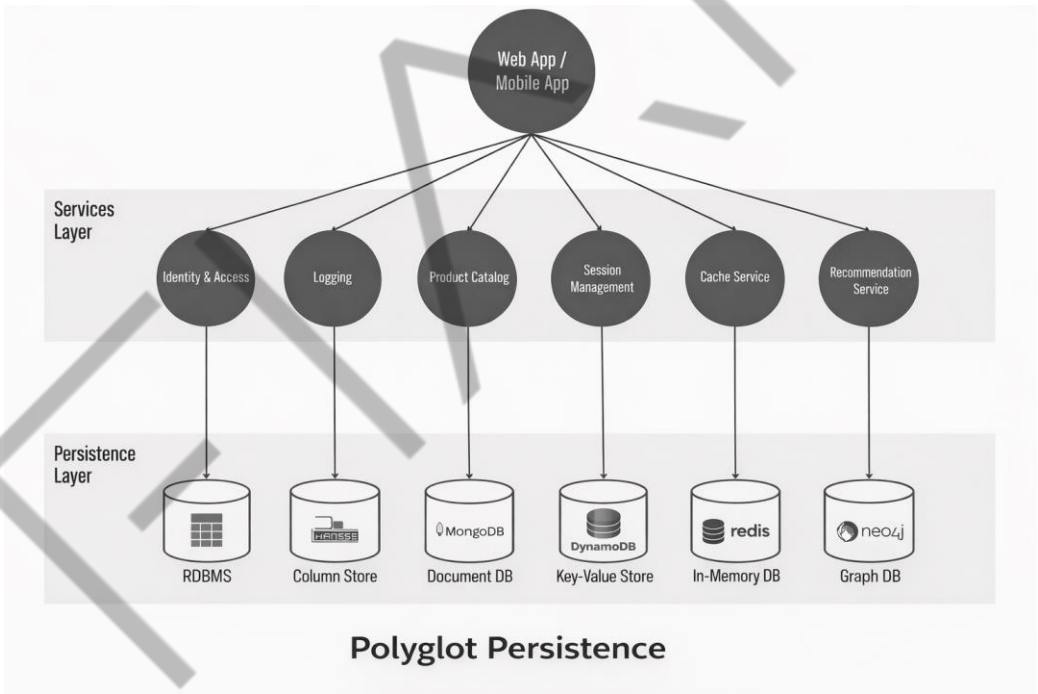


Figura 6 – NoSQL – Polyglot Persistence

Fonte: Databricks, adaptado pelo autor (2026)

Na prática, o NoSQL raramente substitui completamente o modelo relacional. Em arquiteturas modernas, é comum a adoção de **Polyglot Persistence**, combinando diferentes tecnologias de armazenamento, cada uma atendendo melhor a um tipo de necessidade.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

Nesse contexto, o NoSQL atua como:

- Camada operacional de alta escala;
- Fonte de dados para pipelines analíticos;
- Base para aplicações em tempo real;
- Suporte a machine learning e streaming.

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

MERCADO, CASES E TENDÊNCIAS

NoSQL na prática e os rumos do mercado

O NoSQL consolidou-se como um componente estratégico nas arquiteturas modernas de dados, impulsionado pela necessidade de escala, disponibilidade e flexibilidade. Com o crescimento de aplicações digitais, microsserviços e plataformas globais, tornou-se evidente que o modelo relacional, embora essencial, não atende isoladamente a todos os cenários. O mercado passou a adotar o NoSQL de forma pragmática, direcionando seu uso para contextos onde baixa latência, alta concorrência e dados heterogêneos são requisitos centrais.

Cases representativos

Empresas digitais de grande escala utilizam NoSQL como base de seus sistemas críticos. A Netflix emprega bancos NoSQL para gerenciar sessões, metadados e recomendações, priorizando disponibilidade e tolerância a falhas. A Amazon, por sua vez, desenvolveu tecnologias próprias para lidar com volumes massivos de dados distribuídos, culminando em soluções como o Amazon DynamoDB, amplamente utilizadas em catálogos, carrinhos de compra e controle de pedidos. Já a Uber opera em um ambiente altamente dinâmico, combinando bancos relacionais, NoSQL e streaming para processar dados em tempo real com baixa latência.

Esses exemplos reforçam que o NoSQL é adotado quando a experiência do usuário e a continuidade do serviço são mais críticas do que a consistência imediata.

Tendências consolidadas

Uma das principais tendências do mercado é a adoção da Polyglot Persistence, na qual diferentes bancos coexistem na mesma arquitetura, cada um resolvendo melhor um tipo específico de problema. Além disso, observa-se o crescimento acelerado de soluções NoSQL gerenciadas em nuvem, como MongoDB Atlas e Azure Cosmos DB, reduzindo o esforço operacional e permitindo maior foco no negócio.

Outra tendência relevante é a integração do NoSQL com arquiteturas analíticas modernas, como Data Lakes e Lakehouses, ampliando seu papel para

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

além do armazenamento operacional e aproximando-o de análises em tempo real e machine learning.

Visão estratégica

O mercado demonstra que o NoSQL não substitui o modelo relacional, mas o complementa. A maturidade arquitetural está em compreender os trade-offs envolvidos e em selecionar a tecnologia mais adequada para cada contexto. Em um cenário cada vez mais distribuído e orientado a serviços, o NoSQL se estabelece como peça fundamental para a construção de sistemas escaláveis, resilientes e alinhados às demandas reais do negócio.

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, você foi introduzido(a) aos **fundamentos do NoSQL**, compreendendo seu surgimento, seus principais conceitos e as diferenças essenciais em relação ao **modelo relacional**. O conteúdo destacou que o NoSQL não representa uma substituição direta dos bancos SQL, mas sim uma **evolução do ecossistema de dados**, criada para atender às demandas de escala, disponibilidade e flexibilidade das aplicações modernas.

Você analisou como o **modelo relacional** se baseia em estruturas rígidas, normalização e consistência forte (ACID), sendo altamente eficiente para sistemas transacionais tradicionais. Em contraste, explorou como o **NoSQL** adota modelos de dados variados, schemas flexíveis e uma abordagem orientada ao padrão de acesso, priorizando desempenho e escalabilidade horizontal.

Outro ponto central foi a compreensão dos **sistemas distribuídos**, incluindo conceitos como **replicação, sharding e escalabilidade horizontal**, além do impacto do **Teorema CAP** nas decisões arquiteturais. Foi possível entender por que muitos bancos NoSQL optam por consistência eventual como um trade-off consciente para garantir alta disponibilidade e tolerância a falhas.

A aula também apresentou os **principais tipos de bancos NoSQL** — documentos, chave-valor, colunas e grafos — e discutiu seus contextos de aplicação, vantagens e limitações. Por fim, você consolidou a visão de que arquiteturas modernas adotam o conceito de **Polyglot Persistence**, combinando bancos relacionais e NoSQL de forma estratégica, conforme as necessidades do negócio.

Ao final desta aula, você deve ser capaz de **avaliar criticamente** quando utilizar bancos relacionais ou NoSQL, compreendendo os impactos técnicos e arquiteturais dessa escolha em sistemas reais.

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

REFERÊNCIAS

AMAZON WEB SERVICES. What Is NoSQL?. Disponível em: https://aws.amazon.com/nosql/ Acesso em: 26 mar. 2026.

BREWER, Eric A. Towards Robust Distributed Systems. Proceedings of the Annual ACM Symposium on Principles of Distributed Computing, 2000.

DATABRICKS. What is NoSQL?. Disponível em: https://www.databricks.com/glossary/nosql Acesso em: 26 mar. 2026.

GOOGLE. Bigtable: A Distributed Storage System for Structured Data. Disponível em: https://research.google/pubs/pub27898/ Acesso em: 26 mar. 2026.

KLEPPMANN, Martin. Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. Sebastopol: O'Reilly Media, 2017.

MONGODB. Introduction to NoSQL Databases. Disponível em: https://www.mongodb.com/resources/basics/databases/nosql-explained
Acesso em: 26 mar. 2026.

SADALAGE, Pramod J.; FOWLER, Martin. NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence. Boston: Addison-Wesley, 2013.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

PALAVRAS-CHAVE

NoSQL. Escalabilidade Horizontal. Sistemas Distribuídos

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com



PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com